arXiv:2610.31642v1 [cs.LG] 22 Oct 2026

A Fix for Softmax and Cross-Entropy

Jonathan T. Barron
Google DeepMind
barron@google.com

Dan Gnanapragasam
Google DeepMind
nanabyte@google.com

Michael Tschannen
Google DeepMind
tschannen@google.com

Abstract

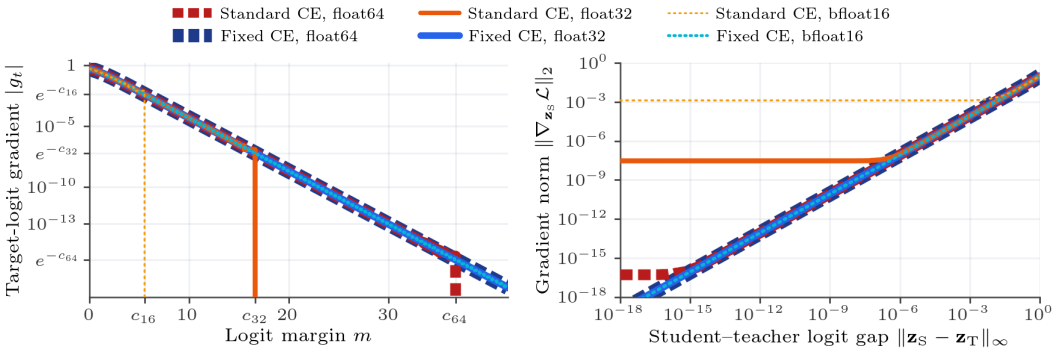
Standard implementations of softmax, log-softmax, and cross-entropy have two numerical bugs. The first bug, *Softmax Collapse*, rounds the target logit's gradient to zero if that one logit is much larger than the others, resulting in gradients that no longer sum to zero. The second bug, *Softmax Cancellation*, occurs when student logits are close to teacher logits, and produces non-zero gradients even when the student and teacher logits are equal. We present simple implementations of log-softmax, softmax, and cross-entropy that fix both bugs. Our fix eliminates the previously-observed "Slingshot" phenomenon, speeds up grokking with weight decay, prevents accuracy collapse without weight decay, stops student networks from drifting away from the teacher during distillation, and shows that after memorization, logit growth is bounded by an equilibrium between weight decay and Adam's ϵ hyperparameter.

1 Introduction

Naively computing softmax or cross-entropy on logits $\mathbf{z} \in \mathbb{R}^K$ as $e^{z_k} / \sum_j e^{z_j}$ overflows if any logit exceeds the floating-point exponent limit (≈ 88 in fp32), which is why standard implementations in all deep learning frameworks (JAX, PyTorch, TensorFlow, etc.) shift the input logits by subtracting their maximum value $z_{\max} = \max_j z_j$ (Higham, 2002):

$$\tilde{z}_k = z_k - z_{\max}\,, \quad \text{logsoftmax}(\mathbf{z})_k = \tilde{z}_k - \log \sum_j e^{\tilde{z}_j}\,, \quad q_k = \text{softmax}(\mathbf{z})_k = \frac{e^{\tilde{z}_k}}{\sum_j e^{\tilde{z}_j}}\,. \tag{1}$$

For a target distribution $\mathbf{p} \in \Delta^{K-1}$, cross-entropy is then computed as $\mathcal{L}(\mathbf{z}, \mathbf{p}) = -\sum_k p_k \, \text{logsoftmax}(\mathbf{z})_k$, whose exact gradient is $\mathbf{g} = \nabla_{\mathbf{z}} \mathcal{L}(\mathbf{z}, \mathbf{p}) = \mathbf{q} - \mathbf{p}$. Though subtracting $z_{\max}$ prevents overflow, two numerical bugs remain.



(a) Softmax Collapse: $|g_t|$ vs. margin m.

(b) Softmax Cancellation: $\|\nabla_{\mathbf{z}_S} \mathcal{L}\|_2$ as $\mathbf{z}_S \to \mathbf{z}_T$.

Figure 1: The two bugs in standard softmax and cross-entropy. **(a) Softmax Collapse:** As a model becomes more confident in the correct class (thereby increasing the logit margin m), the true gradient on the target logit shrinks exponentially, but standard implementations incorrectly round that gradient to zero once m crosses a precision cliff (≈ 5.5 in bf16, 16.6 in fp32, and 36.7 in fp64). **(b) Softmax Cancellation:** In soft-target distillation, as student logits $\mathbf{z}_S$ approach teacher logits $\mathbf{z}_T$, the true gradient shrinks toward zero, but standard implementations lose precision and plateau at a non-zero noise floor. Our fix eliminates both bugs and yields accurate gradients across all precisions.

The first bug, *Softmax Collapse* (Figure 1a), occurs when a model correctly predicts the target class $t \in \{1, \ldots, K\}$ with high confidence, so that $z_t = z_{\max}$ is much larger than all non-target logits z_j ($j \neq t$). We measure this confidence by the *logit margin*, which is how much larger the target logit is than the log-sum-exp of all non-target logits:

$$m = z_t - \log \sum_{j \neq t} e^{z_j} \,. \tag{2}$$

In Equation 1, subtracting $z_{\max} = z_t$ sets the shifted target logit to $\tilde{z}_t = 0$ (so $e^{\tilde{z}_t} = 1$ in the denominator), while the $K - 1$ non-target exponentials sum to a small positive number $e^{-m} = \sum_{j \neq t} e^{z_j - z_t} \ll 1$. Floating-point numbers have much finer precision near 0 than near 1: in fp32, values near 0 can be as small as $\sim 10^{-38}$, whereas values added to 1.0 are lost to rounding below $\sim 10^{-7}$. Because standard implementations form the denominator $1 + e^{-m}$ by adding e^{-m} to 1.0, the e^{-m} term is lost to rounding once the margin m exceeds a threshold c_d that we call the *precision cliff* ($c_{16} \approx 5.5$ in bf16, $c_{32} \approx 16.6$ in fp32, and $c_{64} \approx 36.7$ in fp64). When $m > c_d$, the denominator $1 + e^{-m}$ evaluates to exactly 1.0, which causes $q_t = 1/(1 + e^{-m})$ to evaluate to 1.0 and zeroes out both the loss and the target gradient $\tilde{g}_t = q_t - 1 = 0.0$. However, the non-target gradients $\tilde{g}_j = q_j = e^{z_j - z_t}/(1 + e^{-m})$ ($j \neq t$) are evaluated near 0 without adding 1.0 to their numerators, so they remain positive all the way down to exponent underflow ($m \approx 88$). Because the target gradient rounds to 0.0 while the non-target gradients do not, the gradients no longer sum to zero ($\sum_k \tilde{g}_k > 0$). When training with adaptive optimizers such as Adam and AdamW (Kingma & Ba, 2015; Loshchilov & Hutter, 2019), which normalize each gradient by a moving average of its magnitude, this asymmetry causes the recurring "Slingshot" loss spikes observed by Thilak et al. (2022) and the grokking failures studied by Prieto et al. (2025). While prior works used surrogate losses (Prieto et al., 2025), gradient projection (Liu et al., 2026), or fp64 casting (Nanda et al., 2023; Varma et al., 2023) to mitigate these issues, we instead fix log-softmax, softmax, and cross-entropy directly (Section 2).

The second bug, *Softmax Cancellation* (Figure 1b), occurs in knowledge distillation (Hinton et al., 2015) and KL regularization, where the target $\mathbf{p} = \mathrm{softmax}(\mathbf{z}_\mathrm{T})$ comes from teacher logits $\mathbf{z}_\mathrm{T}$. Standard implementations compute $\mathbf{q} = \mathrm{softmax}(\mathbf{z}_\mathrm{S})$ and $\mathbf{p} = \mathrm{softmax}(\mathbf{z}_\mathrm{T})$ separately and subtract them to get $\tilde{\mathbf{g}} = \mathbf{q} - \mathbf{p}$. This means that if $\mathbf{z}_\mathrm{S}$ is close to $\mathbf{z}_\mathrm{T}$, subtracting these nearly equal probabilities loses precision, and slight differences between the forward-pass softmax and backward-pass log-softmax formulas leave a non-zero gradient even when $\mathbf{z}_\mathrm{S} = \mathbf{z}_\mathrm{T}$. We fix this by computing $\mathbf{q} - \mathbf{p}$ directly from $\mathbf{z}_\mathrm{S} - \mathbf{z}_\mathrm{T}$ in log space (Section 3).

2 Softmax Collapse

When the target logit $z_t = z_{\max}$ is the largest logit, shifting by $z_{\max}$ in Equation 1 sets $\tilde{z}_t = 0$ and $\tilde{z}_j = z_j - z_t < 0$ for $j \neq t$, so the denominator is $\sum_i e^{\tilde{z}_i} = 1 + \sum_{j \neq t} e^{z_j - z_t} = 1 + e^{-m}$. Mathematically, the hard-target loss is $\mathcal{L}(\mathbf{z}, t) = -\log\mathrm{softmax}(\mathbf{z})_t = \log(1 + e^{-m})$ and its gradients are

$$g_t = \underbrace{\frac{1}{1 + e^{-m}} - 1}_{\text{Standard}} = \underbrace{-\frac{e^{-m}}{1 + e^{-m}}}_{\text{Fixed}}, \qquad g_j = \frac{e^{z_j - z_t}}{1 + e^{-m}} \quad \text{for } j \neq t \,, \tag{3}$$

which sum to zero ($\sum_k g_k = 0$). In floating point, however, the two expressions for g_t in Equation 3 behave very differently once $m > c_d$, where e^{-m} is too small to change 1.0 when added to it (Figure 1a). Standard implementations add 1 and e^{-m} to form the denominator $1 + e^{-m}$, which rounds to 1 when $m > c_d$. In the forward pass, $\log(1 + e^{-m})$ then evaluates to $\log(1) = 0$, zeroing out both $\mathrm{logsoftmax}(\mathbf{z})_t$ and $\mathcal{L}(\mathbf{z}, t)$. In the backward pass, standard cross-entropy implementations compute $q_t = 1/(1 + e^{-m}) = 1$ and subtract 1 to get $\tilde{g}_t = q_t - 1 = 0$. In contrast, our fixed expression $-e^{-m}/(1 + e^{-m})$ keeps e^{-m} in the numerator: even when the denominator $1 + e^{-m}$ rounds to 1.0, dividing $-e^{-m}$ by 1.0 preserves $g_t \approx -e^{-m}$ down to exponent underflow ($m \approx 88$).

While $\tilde{g}_t$ rounds to zero when $m > c_d$, the non-target gradients do not. For each non-target class $j \neq t$, standard cross-entropy computes $\tilde{g}_j = q_j = e^{z_j - z_t}/(1 + e^{-m})$. Because the numerator $e^{z_j - z_t}$ is evaluated directly from the negative difference $z_j - z_t$ without adding 1, it remains accurate down to exponent underflow while the denominator rounds to 1. When $m > c_d$, standard cross-entropy therefore zeroes out only the target entry t while all $K - 1$ non-target gradients remain positive:

$$\tilde{g}_k = \mathbf{1}[k \neq t]\, g_k \,, \qquad \sum_k \tilde{g}_k = -g_t = \frac{e^{-m}}{1 + e^{-m}} > 0 \quad \text{for } m > c_d \,. \tag{4}$$

The same bug affects standalone $\mathrm{softmax}(\mathbf{z})$: once $m > c_d$, softmax's backward pass rounds $q_{i_{\max}}$ ($i_{\max} = \arg\max_i z_i$) to 1 and sets $\tilde{g}_{i_{\max}} = 0$ while the remaining gradients stay non-zero.

This bug can be fixed by never adding 1 to e^{-m} directly. In $\mathrm{logsoftmax}(\mathbf{z})$ and $\mathrm{softmax}(\mathbf{z})$, only the largest logit $i_{\max} = \arg\max_i z_i$ is shifted to 0 (where $e^0 = 1$ in the denominator). Instead of including $e^0 = 1$ in the sum of exponentials, we set $\tilde{z}_{i_{\max}} = 0$, compute the margin m by excluding $i_{\max}$ from the log-sum-exp, and combine them with a numerically stable $\mathrm{softplus}(x) = \log(1 + e^x)$ (with $\mathrm{softmax}(\mathbf{z})_k = \exp(\mathrm{logsoftmax}(\mathbf{z})_k)$):

$$\tilde{z}_k = \mathbf{1}[k \neq i_{\max}](z_k - z_{i_{\max}}), \quad m = -\operatorname*{LSE}_{j \neq i_{\max}} \tilde{z}_j, \quad \mathrm{logsoftmax}(\mathbf{z})_k = \tilde{z}_k - \mathrm{softplus}(-m). \tag{5}$$

For hard-target cross-entropy with target t, we can use t directly in place of $i_{\max}$ without computing an $\arg\max$:

$$m = -\operatorname*{LSE}_{j \neq t}(z_j - z_t), \qquad \mathcal{L}(\mathbf{z}, t) = \log(1 + e^{-m}) = \mathrm{softplus}(-m). \tag{6}$$

Because the t-th slot is replaced with the constant 0 rather than $z_t - z_t$, z_t only appears in $\mathcal{L}$ through the $K - 1$ differences $z_j - z_t$ ($j \neq t$), so autodiff correctly computes the target gradient as the negated sum of the non-target gradients:

$$g_j = e^{z_j - z_t - \mathcal{L}} = \frac{e^{z_j - z_t}}{1 + e^{-m}} \quad \text{for } j \neq t, \qquad g_t = -\sum_{j \neq t} g_j = -\frac{e^{-m}}{1 + e^{-m}}. \tag{7}$$

Here is a JAX implementation of $\mathrm{logsoftmax}$, $\mathrm{softmax}$, and hard-target cross-entropy, which remains accurate down to exponent underflow and satisfies $\sum_k g_k = 0$:

```
import jax, jax.numpy as jnp, optax

def log_softmax(z):
  i_max = jnp.argmax(z, axis=-1)
  z_max = jnp.take_along_axis(z, i_max[..., None], axis=-1)  # Max logit.
  is_max = jax.nn.one_hot(i_max, z.shape[-1], dtype=bool)    # Max mask.
  shifted = jnp.where(is_max, 0.0, z - z_max)                # Exact 0 at argmax.
  others = jnp.where(is_max, -jnp.inf, shifted)              # Exclude argmax from sum.
  m = -jax.nn.logsumexp(others, axis=-1, keepdims=True)      # Logit margin.
  return shifted - jax.nn.softplus(-m)                       # log_softmax(z).

softmax = lambda z: jnp.exp(log_softmax(z))

def cross_entropy(z, t):
  z_t = jnp.take_along_axis(z, t[..., None], axis=-1)        # Target logit.
  is_t = jax.nn.one_hot(t, z.shape[-1], dtype=bool)          # Target mask.
  others = jnp.where(is_t, -jnp.inf, z - z_t)                # Exclude target from sum.
  m = -jax.nn.logsumexp(others, axis=-1)                     # Logit margin.
  return jax.nn.softplus(-m)                                 # -log_softmax(z)_t.
```

As an example, consider $\mathbf{z} = [9.0, -9.0, -9.0]$ and $t = 0$. The standard `jax.nn` and `optax` implementations return zero for the target log-probability, loss, and target gradient, while our functions return accurate values whose gradients sum to zero:

```
z, t = jnp.array([9.0, -9.0, -9.0], dtype=jnp.float32), jnp.array(0)
std_ce = optax.softmax_cross_entropy_with_integer_labels
jax.nn.log_softmax(z)                       # ->           [ 0.0,      -18.0,     -18.0    ]
jax.grad(lambda z: jax.nn.softmax(z)[0])(z) # ->           [ 0.0,      -1.523e-8, -1.523e-8]
jax.value_and_grad(std_ce)(z, t)            # -> 0.0,      [ 0.0,       1.523e-8,  1.523e-8]
log_softmax(z)                              # ->           [-3.046e-8, -18.0,     -18.0    ]
jax.grad(lambda z: softmax(z)[0])(z)        # ->           [ 3.046e-8, -1.523e-8, -1.523e-8]
jax.value_and_grad(cross_entropy)(z, t)     # -> 3.046e-8, [-3.046e-8,  1.523e-8,  1.523e-8]
```

3 SOFTMAX CANCELLATION

Given student logits $\mathbf{z}_\mathrm{S} \in \mathbb{R}^K$ and teacher logits $\mathbf{z}_\mathrm{T} \in \mathbb{R}^K$, standard soft-target cross-entropy computes $\mathbf{p} = \mathrm{softmax}(\mathbf{z}_\mathrm{T})$ and evaluates

$$\mathcal{L}(\mathbf{z}_\mathrm{S}, \mathbf{z}_\mathrm{T}) = -\sum_k p_k \ \mathrm{logsoftmax}(\mathbf{z}_\mathrm{S})_k, \quad \mathbf{q} = \mathrm{softmax}(\mathbf{z}_\mathrm{S}), \quad \tilde{g}_k = \frac{\partial \mathcal{L}}{\partial z_{\mathrm{S},k}} = q_k - p_k. \tag{8}$$

As the student logits approach the teacher logits ($\mathbf{z}_\mathrm{S} \to \mathbf{z}_\mathrm{T}$), the true gradient $\mathbf{g} = \mathbf{q} - \mathbf{p}$ shrinks toward $\mathbf{0}$ and equals $\mathbf{0}$ when $\mathbf{z}_\mathrm{S} = \mathbf{z}_\mathrm{T}$. In floating point, however, computing $\mathbf{q}$ and $\mathbf{p}$ separately and subtracting them (Equation 8) loses precision when $\mathbf{z}_\mathrm{S}$ is close to $\mathbf{z}_\mathrm{T}$, because the small logit difference $\boldsymbol{\delta} = \mathbf{z}_\mathrm{S} - \mathbf{z}_\mathrm{T}$ is lost when $\mathbf{q}$ and $\mathbf{p}$ are rounded to probabilities before being subtracted (Figure 1b). Worse, because $\mathbf{p} = \mathrm{softmax}(\mathbf{z}_\mathrm{T})$ in the forward pass and $\mathbf{q} = \exp(\mathrm{logsoftmax}(\mathbf{z}_\mathrm{S}))$ in the backward pass are evaluated using slightly different sequences of operations, $\mathbf{q}$ and $\mathbf{p}$ do not match to machine precision even when $\mathbf{z}_\mathrm{S} = \mathbf{z}_\mathrm{T}$, leaving a non-zero gradient $\tilde{\mathbf{g}} = \mathbf{q} - \mathbf{p} \neq \mathbf{0}$.

To avoid subtracting nearly equal probabilities, we pass $\mathbf{z}_\mathrm{T}$ directly to our loss function and compute $\mathbf{q} - \mathbf{p}$ in log space using $\mathrm{expm1}(x) = e^x - 1$ and $\mathrm{log1p}(x) = \log(1 + x)$. Given $\boldsymbol{\delta} = \mathbf{z}_\mathrm{S} - \mathbf{z}_\mathrm{T}$, we center $\boldsymbol{\delta}$ under $\mathbf{p}$ and write the log-ratio $s_k = \log(q_k/p_k)$ and difference $u_k = q_k - p_k$ as:

$$d_k = \delta_k - \sum_i p_i \delta_i \,, \qquad s_k = d_k - \mathrm{log1p}\left(\sum_i p_i \,\mathrm{expm1}(d_i)\right), \qquad u_k = p_k \,\mathrm{expm1}(s_k) \,. \quad (9)$$

Because $\sum_i p_i d_i = 0$ and $\mathrm{expm1}(x) \geq x$, the input to $\mathrm{log1p}$ is always non-negative. We then set $g_k = u_k - q_k \sum_i u_i$ to remove any remaining floating-point drift from $\sum_k g_k = 0$. Because d_k, s_k, and u_k are evaluated using $\mathrm{expm1}$ and $\mathrm{log1p}$ without adding or subtracting 1, $\mathbf{g}$ remains accurate as $\mathbf{z}_\mathrm{S} \to \mathbf{z}_\mathrm{T}$ and is $\mathbf{0}$ when $\mathbf{z}_\mathrm{S} = \mathbf{z}_\mathrm{T}$. Here is a JAX implementation of soft-target cross-entropy (using `log_softmax` and `softmax` from Section 2), which uses `jax.lax.stop_gradient` (`sg`) to return the forward loss while substituting $\mathbf{g}$ in the backward pass:

```
def soft_target_cross_entropy(z_S, z_T):
  log_q = log_softmax(z_S)
  q, p = jnp.exp(log_q), softmax(z_T)
  delta = z_S - z_T                                                  # Logit difference.
  d = delta - jnp.sum(p * delta, axis=-1, keepdims=True)             # Center under p.
  s = d - jnp.log1p(jnp.sum(p * jnp.expm1(d), axis=-1, keepdims=True))  # log(q / p).
  u = p * jnp.expm1(s)                                               # Difference q - p.
  g = u - q * jnp.sum(u, axis=-1, keepdims=True)                     # Ensure sum(g) = 0.
  sg = jax.lax.stop_gradient
  loss = -jnp.sum(p * sg(log_q), axis=-1)                            # Forward loss.
  return loss + jnp.sum(sg(g) * (z_S - sg(z_S)), axis=-1)            # Custom gradient g.
```

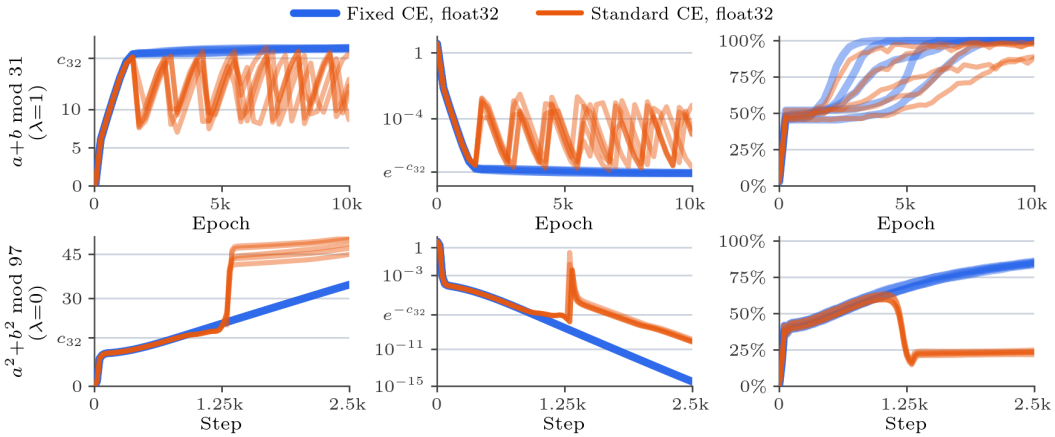
As an example, let $\mathbf{z}_\mathrm{S}$ be a small fp32 perturbation of $\mathbf{z}_\mathrm{T}$. Standard soft-target cross-entropy shrinks the second gradient entry by $3\times$, flips the sign of the third, and fails to sum to zero, while our implementation matches the exact fp64 gradient:

```
z_T = jnp.array([2.0, 1.0, 0.0], dtype=jnp.float32)
z_S = z_T + jnp.array([-1e-7, 1e-7, 0.0], dtype=jnp.float32)
std_soft_ce = lambda z_S, z_T: -jnp.sum(jax.nn.softmax(z_T) * jax.nn.log_softmax(z_S))
jax.grad(std_soft_ce)(z_S, z_T)               # -> [-5.960464e-8, 1.451589e-8, -1.123343e-9]
jax.grad(soft_target_cross_entropy)(z_S, z_T) # -> [-4.595504e-8, 4.144190e-8,  4.513142e-9]
# Exact (float64) gradient:                      [-4.595504e-8, 4.144190e-8,  4.513142e-9]
```

4 The Slingshot Mechanism

In grokking (Power et al., 2022), a network quickly memorizes the training set ($q_t \to 1$) and generalizes much later. While studying grokking on modular addition with Adam (Kingma & Ba, 2015; Loshchilov & Hutter, 2019), Thilak et al. (2022) observed the *Slingshot Mechanism*, where training after memorization alternates between flat loss plateaus near zero and sudden loss spikes that coincide with jumps in test accuracy.

These Slingshot spikes are a direct consequence of Softmax Collapse (Section 2). Recall that AdamW updates parameters $\boldsymbol{\theta}$ by subtracting $\eta\big(\hat{\mathbf{m}}/(\sqrt{\hat{\mathbf{v}}} + \epsilon) + \lambda\boldsymbol{\theta}\big)$, where $\hat{\mathbf{m}}$ and $\hat{\mathbf{v}}$ are bias-corrected moving averages of the parameter gradient $\mathbf{g} = \nabla_{\boldsymbol{\theta}}\mathcal{L}(\boldsymbol{\theta})$ and its square (with decay rates β_1, β_2). After memorization, the logit margins m grow as the network pushes its target probabilities toward 1. Once an example's margin exceeds the precision cliff c_d, standard cross-entropy rounds its target gradient $\tilde{g}_t$ to zero while its non-target gradients $\tilde{g}_j > 0$ ($j \neq t$) remain positive. When all examples in a batch sit above c_d, the parameter gradients nearly vanish and Adam's second moment $\hat{v}_i$ decays exponentially toward zero while weight decay slowly shrinks the parameters. Eventually, weight decay or the residual non-target updates pull a few examples back below c_d, where their target



(a) Mean logit margin m. (b) Training loss. (c) Test accuracy.

Figure 2: The Slingshot mechanism (Thilak et al., 2022) with and without weight decay (5 random seeds, fp32). **Top row ($a + b \bmod 31$, $\lambda = 1$, $\epsilon = 10^{-8}$, Varma et al. 2023):** Standard cross-entropy hits the fp32 cliff ($m = c_{32} \approx 16.6$, where loss rounds to 0) and repeatedly drops and rebounds, slowing and degrading grokking; fixed cross-entropy crosses c_{32} smoothly, groks faster on all 5 seeds, and saturates just above c_{32} at $m \approx 18.4$ (Section 5). **Bottom row ($a^2 + b^2 \bmod 97$, $\lambda = 0$, $\epsilon = 10^{-16}$, Doshi et al. 2024):** Without weight decay, hitting c_{32} around step 1250 triggers a spike that drops standard cross-entropy test accuracy from 61% to 23%, whereas fixed cross-entropy continues smoothly to 85%.

gradients $\tilde{g}_t \approx -e^{-m}$ abruptly turn back on. Because $\sqrt{\hat{v}_i}$ decayed while those examples were above c_d, Adam divides the restored gradients by a very small denominator $\sqrt{\hat{v}_i} + \epsilon$, producing a large parameter step that spikes the training loss, resets $\hat{v}_i$, and repeats the cycle. With our fixed cross-entropy, the target-logit gradient $g_t = -e^{-m}/(1 + e^{-m})$ does not round to zero at c_d, so $\hat{m}_i$ and $\sqrt{\hat{v}_i}$ decay smoothly together and Slingshot spikes disappear (Figure 2).

Whether Slingshot spikes appear to help or hurt generalization depends on how the network's weight norm is controlled (Figure 2). When a network uses LayerNorm before the output layer and no weight decay (Thilak et al., 2022; Prieto et al., 2025), growing the pre-LayerNorm weights does not change the logits, but it does shrink the effective step size $\eta/\|\boldsymbol{\theta}\|_2$; each Slingshot spike inflates $\|\boldsymbol{\theta}\|_2$ and acts as an accidental learning-rate decay. When weight norms are already controlled by weight decay (Varma et al., 2023), Slingshot spikes at c_{32} only slow down and hurt grokking ($94 \pm 5\%$ vs. $100 \pm 0\%$ test accuracy on $a + b \bmod 31$; Figure 2, top). And when neither LayerNorm nor weight decay is used (Doshi et al., 2024), a single Slingshot spike at c_{32} is destructive: on $a^2 + b^2 \bmod 97$ ($\lambda = 0$, $\epsilon = 10^{-16}$), standard cross-entropy tracks fixed cross-entropy up to 61% test accuracy until m hits c_{32} around step 1250, where a spike drops test accuracy to 23% (and from 60% to 21% on $a^3 + b \bmod 97$), while fixed cross-entropy continues smoothly to 85% (Figure 2, bottom).

5 Margin Equilibria

In Figure 2a, our fixed cross-entropy eliminates the precision cliff at $c_{32} \approx 16.6$, yet why does its margin still saturate just above c_{32} at $m \approx 18.4$ when weight decay is applied ($\lambda = 1$, top) while continuing to climb without weight decay ($\lambda = 0$, bottom)? This plateau has nothing to do with fp32 precision; rather, it is the equilibrium where weight decay balances AdamW's normalized gradient step. At a stationary point where the parameters stop changing, the moving averages converge to $\hat{m}_i = g_i$ and $\hat{v}_i = g_i^2$ (so $\sqrt{\hat{v}_i} = |g_i|$), and setting the parameter update to zero gives:

$$\frac{\hat{m}_i}{\sqrt{\hat{v}_i} + \epsilon} + \lambda\theta_i = 0 \quad \Longrightarrow \quad \frac{g_i}{|g_i| + \epsilon} + \lambda\theta_i = 0 \quad \forall i\,. \tag{10}$$

Here η, β_1, and β_2 cancel out, so RMSProp (Tieleman & Hinton, 2012) and LaProp (Ziyin et al., 2020) share the same stationary point. After memorization, the gradients shrink exponentially with the margin ($|g_i| \propto e^{-m}$). If $\epsilon = 0$, the normalized gradient $g_i/|g_i| = \mathrm{sign}(g_i)$ has a magnitude of 1

no matter how small $|g_i|$ becomes, which causes the margin to grow until the gradients underflow. When $\epsilon > 0$, however, the normalized gradient has magnitude ≈ 1 while $|g_i| \gg \epsilon$, and shrinks as $g_i/\epsilon \propto e^{-m}$ once $|g_i| \ll \epsilon$ (Figure 3). Balancing this shrinking step against weight decay $\lambda\theta_i$ requires $|g_i| \approx \epsilon\lambda|\theta_i|$ (Wortsman et al., 2024; Molybog et al., 2023; Everett et al., 2024; Kosson et al., 2024), which acts like an L_2 regularizer with weight $\epsilon\lambda$ (Rosset et al., 2004; Soudry et al., 2018). Since $|g_i| \propto e^{-m}$, the margin stops growing at:

$$m^\star(\epsilon) \approx \log(1/\epsilon)\,. \tag{11}$$

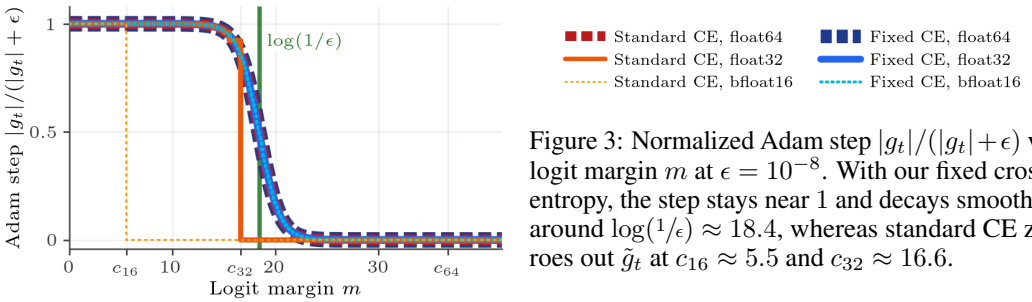


Figure 3: Normalized Adam step $|g_t|/(|g_t|+\epsilon)$ vs. logit margin m at $\epsilon = 10^{-8}$. With our fixed cross-entropy, the step stays near 1 and decays smoothly around $\log(1/\epsilon) \approx 18.4$, whereas standard CE zeroes out $\tilde{g}_t$ at $c_{16} \approx 5.5$ and $c_{32} \approx 16.6$.

Equation 11 explains why the default $\epsilon = 10^{-8}$ causes Slingshot in fp32: its equilibrium $\log(1/\epsilon) \approx 18.4$ sits just above the fp32 cliff $c_{32} \approx 16.6$ (Figure 3), so standard cross-entropy hits the cliff right before reaching equilibrium (whereas for $\epsilon \geq 10^{-7}$, $\log(1/\epsilon) \leq 16.1 < c_{32}$ and standard cross-entropy settles without spiking). To verify Equation 11 across many orders of magnitude down to $\epsilon = 0$ in fp32 (Figure 4), we use LaProp and update $\sqrt{\hat{\mathbf{v}}}$ directly without squaring g_i^2 (which would otherwise underflow in fp32 once $|g_i| < 2^{-63}$, or $m \approx 38$).

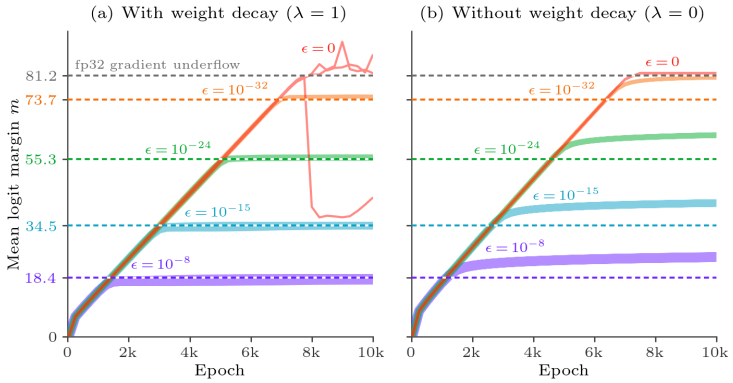
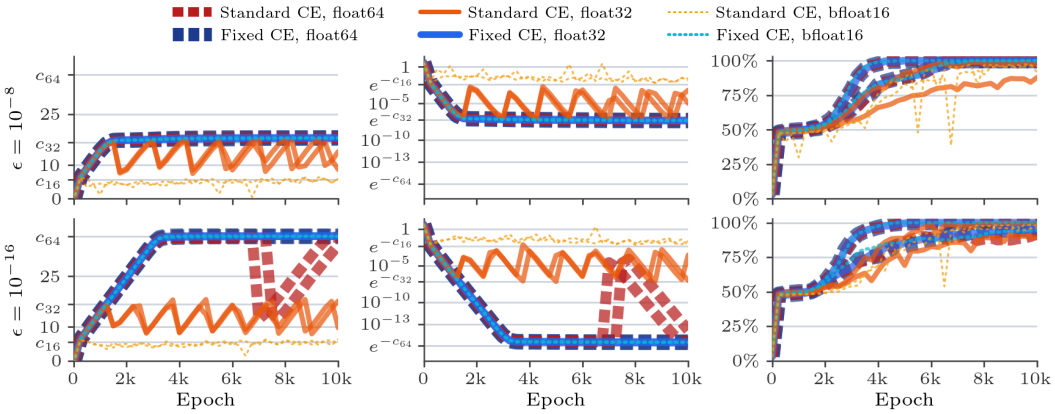


Figure 4: Logit margins when training with our fixed cross-entropy and LaProp on $a + b \bmod 31$ in fp32 ($\beta_1 = 0.9$, $\beta_2 = 0.98$; 3 seeds per setting as solid lines, analytical predictions as dashed lines). **(a)** With weight decay ($\lambda = 1$), the margin for each $\epsilon > 0$ grows along the $\epsilon = 0$ trajectory and levels off at the predicted equilibrium $m^\star(\epsilon) = \log(1/\epsilon)$ (colored y-ticks), whereas $\epsilon = 0$ keeps growing until the fp32 gradients themselves underflow at $m \approx 81.2$ (gray). **(b)** Without weight decay ($\lambda = 0$), nothing balances the g_i/ϵ step, so the margins rise past $\log(1/\epsilon)$ and continue drifting upward.

6 Grokking Across Precisions and Benchmarks

To see how the precision cliff behaves across floating-point formats, we follow Varma et al. (2023) and Lyle et al. (2025) and train a 1-layer width-64 ReLU Transformer on $a + b \bmod 31$ (50% training split) for 10,000 epochs of full-batch AdamW with $\eta = 10^{-3}$, $\beta_1 = 0.9$, $\beta_2 = 0.98$, and $\lambda = 1$. We vary only the loss function (standard vs. fixed cross-entropy), the floating-point precision (bf16, fp32, fp64), and $\epsilon \in \{10^{-8}, 10^{-12}, 10^{-16}\}$ (Figure 5, Table 1). At $\epsilon = 10^{-8}$ (Figure 5, top), standard cross-entropy stalls at $c_{16} \approx 5.5$ in bf16 and repeatedly spikes at $c_{32} \approx 16.6$ in fp32, whereas our fixed cross-entropy crosses both cliffs smoothly to the equilibrium $\log(1/\epsilon) \approx 18.4$ and reaches 100% test accuracy in all three precisions. Reducing ϵ to 10^{-16} (Figure 5, bottom) raises the equilibrium to



(a) Mean logit margin m. (b) Training loss. (c) Test accuracy.

Figure 5: Standard cross-entropy fails at the precision cliff c_d of its floating-point format; fixed cross-entropy removes the cliff across all precisions ($a + b \bmod 31$, $\lambda = 1$; 2 of 5 seeds shown). Top: $\epsilon = 10^{-8}$. Bottom: $\epsilon = 10^{-16}$, where even fp64 standard cross-entropy fails at c_{64} while fixed cross-entropy converges smoothly.

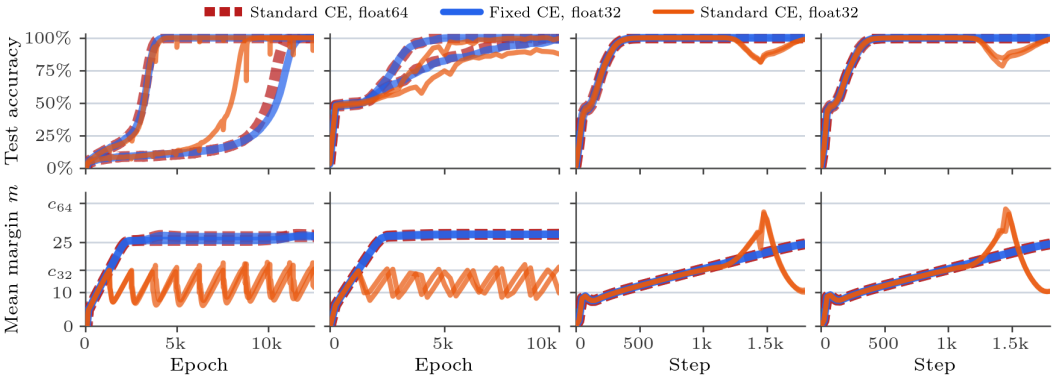
$\log(1/\epsilon) \approx 36.8 > c_{64}$, which causes even fp64 standard cross-entropy to break at $c_{64} \approx 36.7$ while fixed cross-entropy converges smoothly in every precision.

In Table 1 we also compare against two recent methods designed to ameliorate Softmax Collapse by modifying the gradients or the loss function. Liu et al. (2026) subtract the mean logit gradient from each entry so the gradients sum to zero ("Zero-sum Proj."). However, once $m > c_{32}$ and $\tilde{g}_t = 0$, subtracting the mean only gives the target logit $-1/K$ of the total non-target gradient, which stalls the margin near 18 and drops test accuracy below 50% for $\epsilon \leq 10^{-12}$. Prieto et al. (2025) propose "StableMax," which replaces $\exp(x)$ in softmax with a piecewise rational function ($x + 1$ for $x \geq 0$ and $1/(1-x)$ for $x < 0$). Because this function grows linearly instead of exponentially, logit margins blow up to 10^3–10^4 and test accuracy remains lower than with our fixed cross-entropy across all values of ϵ.

Table 1: Comparison of loss functions on modular addition ($a + b \bmod 31$, $\lambda = 1$, 5 seeds) across $\epsilon \in \{10^{-8}, 10^{-12}, 10^{-16}\}$. Fixed CE reaches the predicted equilibrium margin ($\log(1/\epsilon) \approx 18.4, 27.6, 36.8$) and 99–100% test accuracy across all precisions, whereas Standard CE stalls near its precision cliff c_d ($c_{16} \approx 5.5$ in bf16, $c_{32} \approx 16.6$ in fp32, and $c_{64} \approx 36.7$ in fp64).

		Test Accuracy (%)			Final Margin m		
	Variant	$\epsilon = 10^{-8}$	10^{-12}	10^{-16}	10^{-8}	10^{-12}	10^{-16}
fp32	Fixed CE	100.0 ± 0.0	99.6 ± 0.7	99.0 ± 2.3	17.9	27.4	36.8
fp32	Standard CE	94.3 ± 5.4	96.4 ± 5.1	96.5 ± 4.3	13.0	15.1	12.7
bf16	Fixed CE	100.0 ± 0.1	99.8 ± 0.2	98.9 ± 2.4	17.9	27.4	36.8
bf16	Standard CE	93.3 ± 7.4	94.9 ± 4.9	94.2 ± 8.3	5.1	5.6	5.8
fp64	Fixed CE	100.0 ± 0.0	99.5 ± 1.1	99.0 ± 2.0	17.9	27.4	36.8
fp64	Standard CE	100.0 ± 0.0	100.0 ± 0.1	97.2 ± 3.2	17.9	27.4	30.1
fp32	Zero-sum Proj. (Liu et al., 2026)	97.9 ± 3.2	48.9 ± 2.6	49.3 ± 2.5	17.0	18.9	18.9
fp32	StableMax (Prieto et al., 2025)	97.3 ± 4.7	95.6 ± 2.0	97.3 ± 4.3	3200	5661	6313
fp64	StableMax (Prieto et al., 2025)	97.4 ± 4.2	96.7 ± 5.3	95.6 ± 2.7	3170	21252	2322

We observe the same behavior across four grokking benchmarks from the literature at $\epsilon = 10^{-12}$ (Figure 6): 1-layer Transformers on $a + b \bmod 113$ (Nanda et al., 2023) and $a + b \bmod 31$ (Varma et al., 2023) with $\lambda = 1$, and 2-layer MLPs on $a^2 + b^2 \bmod 97$ and $a^3 + b \bmod 97$ (Doshi et al., 2024) with $\lambda = 0.2$. On all four tasks, fp32 standard cross-entropy repeatedly spikes once it hits c_{32}, whereas fp32 fixed cross-entropy matches fp64 standard cross-entropy and reaches 100% test accuracy.

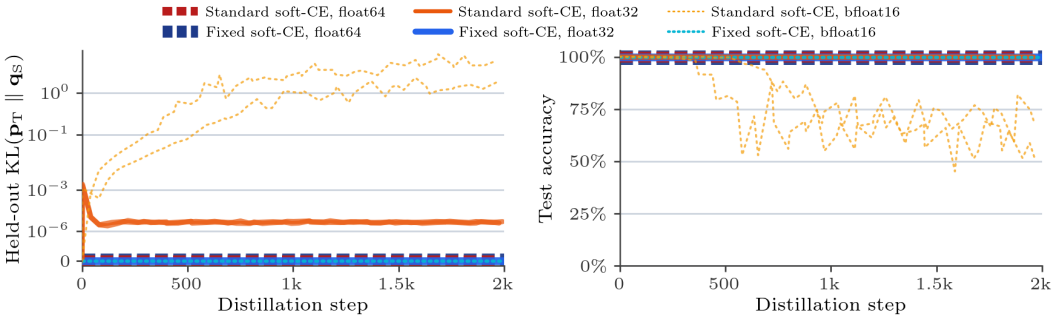


(a) $a+b \bmod 113$ Nanda et al. (2023). (b) $a+b \bmod 31$ Varma et al. (2023). (c) $a^2+b^2 \bmod 97$ Doshi et al. (2024). (d) $a^3+b \bmod 97$ Doshi et al. (2024).

Figure 6: Results across four grokking tasks at $\epsilon = 10^{-12}$ (2 of 3–5 seeds shown): 1-layer Transformers (a, b) and 2-layer MLPs (c, d). Fixed cross-entropy in fp32 (blue) matches fp64 standard cross-entropy (red), while standard cross-entropy in fp32 (orange) breaks at c_{32}.

7 Soft-Target Distillation

To see how Softmax Cancellation (Section 3) affects optimization, we initialize a student network with the weights of a teacher trained to 100% test accuracy on $a + b \bmod 53$ ($\boldsymbol{\theta}_{\mathrm{S}}^{(0)} = \boldsymbol{\theta}_{\mathrm{T}}$), and run 2,000 steps of Adam ($\eta = 2 \times 10^{-3}$, $\lambda = 0$, $\epsilon = 10^{-8}$) at distillation temperature $T = 4$ (Hinton et al., 2015), which keeps the scaled margin m/T below c_{16} so $\mathbf{p}$ does not round to one-hot. Because $\mathbf{z}_{\mathrm{S}} = \mathbf{z}_{\mathrm{T}}$ at initialization, the true gradient is zero and the student should not move. However, standard soft-target cross-entropy has a non-zero residual gradient at $\mathbf{z}_{\mathrm{S}} = \mathbf{z}_{\mathrm{T}}$ due to the mismatch between its forward $\mathrm{softmax}$ and backward $\mathrm{logsoftmax}$ (Figure 1b). Whenever this residual gradient exceeds ϵ, Adam normalizes it into steps of size η (Figure 7): in fp64 the residual ($\sim 10^{-17}$) lies well below $\epsilon = 10^{-8}$ so the student stays near $\mathrm{KL} = 0$, in fp32 ($\sim 10^{-8}$) it causes a brief KL spike, and in bf16 ($\sim 10^{-3}$) it drives the student away from the teacher and degrades test accuracy. Our fixed soft-target cross-entropy returns $\mathbf{g} = \mathbf{0}$ at $\mathbf{z}_{\mathrm{S}} = \mathbf{z}_{\mathrm{T}}$, so the student remains at $\mathrm{KL} = 0$ and 100% test accuracy across all precisions.



(a) Held-out $\mathrm{KL}(\mathbf{p}_{\mathrm{T}} \parallel \mathbf{q}_{\mathrm{S}})$. (b) Test accuracy.

Figure 7: Soft-target distillation when the student is initialized with the teacher's weights ($\boldsymbol{\theta}_{\mathrm{S}}^{(0)} = \boldsymbol{\theta}_{\mathrm{T}}$, $T = 4$, $\epsilon = 10^{-8}$, 2 of 3 seeds shown). Fixed soft-target cross-entropy stays at $\mathrm{KL} = 0$ and 100% accuracy across all precisions; standard soft-target cross-entropy amplifies rounding error into parameter drift in bf16 and a brief KL spike in fp32.

8 Conclusion

We have presented simple fixes for two numerical bugs in standard implementations of softmax, log-softmax, and cross-entropy: *Softmax Collapse*, which zeroes out the dominant logit's gradient at high confidence while leaving non-dominant gradients non-zero, and *Softmax Cancellation*, which

loses precision when student logits approach teacher logits in soft-target distillation. Rewriting these functions in log space using $\mathrm{softplus}$, $\mathrm{expm1}$, and $\mathrm{log1p}$ restores accurate, zero-sum gradients across bf16, fp32, and fp64 without internally casting to higher precision. Fixing these bugs eliminates Slingshot loss spikes during grokking, prevents student networks from drifting away from the teacher during distillation, and reveals that logit margins under AdamW and weight decay naturally converge to $\log(1/\epsilon)$. Though the extreme margins and near-zero student–teacher gaps probed by our experiments are uncommon in standard large-scale training, our fixed functions are easy drop-in replacements that can be implemented without meaningful slowdown, and may prove useful whenever models make high-confidence predictions or distill from close teachers.

ACKNOWLEDGMENTS

We thank Jean-Baptiste Alayrac, Forrester Cole, Roy Frostig, Mark Hamilton, Miki Rubinstein, Keith Rush, and Brandon Zhao for their helpful advice.

REFERENCES

Darshil Doshi, Tianyu He, Aritra Das, and Andrey Gromov. Grokking modular polynomials. *arXiv:2406.03495*, 2024.

Katie Everett, Lechao Xiao, Mitchell Wortsman, Alexander A. Alemi, Roman Novak, Peter J. Liu, Izzeddin Gur, Jascha Sohl-Dickstein, Leslie Pack Kaelbling, Jaehoon Lee, and Jeffrey Pennington. Scaling exponents across parameterizations and optimizers. *ICML*, 2024.

Nicholas J. Higham. *Accuracy and stability of numerical algorithms*. SIAM, 2nd edition, 2002.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv:1503.02531*, 2015.

Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *ICLR*, 2015.

Atli Kosson, Bettina Messmer, and Martin Jaggi. Rotational equilibrium: How weight decay balances learning across neural networks. *ICML*, 2024.

H. Liu, Y. Cao, Z. Li, and Y. Zhou. Grokking or glitching? How low-precision drives Slingshot loss spikes. *arXiv:2605.06152*, 2026.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *ICLR*, 2019.

Clare Lyle, Ghada Sokar, András György, and Razvan Pascanu. What can grokking teach us about learning under non-stationarity? *Conference on Lifelong Learning Agents*, 2025.

Igor Molybog, Peter Albert, Moya Chen, Zachary DeVito, David Esiobu, Naman Goyal, Punit Singh Koura, Sharan Narang, Andrew Poulton, Ruan Silva, et al. A theory on Adam instability in large-scale machine learning. *arXiv:2304.09871*, 2023.

Neel Nanda, Lawrence Chan, Tom Lieberum, Jess Smith, and Jacob Steinhardt. Progress measures for grokking via mechanistic interpretability. *ICLR*, 2023.

Alethea Power, Yuri Burda, Harri Edwards, Igor Babuschkin, and Vedant Misra. Grokking: Generalization beyond overfitting on small algorithmic datasets. *arXiv:2201.02177*, 2022.

Lucas Prieto, Melih Barsbey, Pedro A. M. Mediano, and Tolga Birdal. Grokking at the edge of numerical stability. *ICLR*, 2025.

Saharon Rosset, Ji Zhu, and Trevor Hastie. Margin maximizing loss functions. *NeurIPS*, 2004.

Daniel Soudry, Elad Hoffer, Mor Shpigel Nacson, Suriya Gunasekar, and Nathan Srebro. The implicit bias of gradient descent on separable data. *JMLR*, 2018.

Vimal Thilak, Etai Littwin, Shuangfei Zhai, Omid Saremi, Roei Paiss, and Joshua Susskind. The Slingshot mechanism: An empirical study of adaptive optimizers and the grokking phenomenon. *arXiv:2206.04817*, 2022.

Tijmen Tieleman and Geoffrey Hinton. Lecture 6.5—RMSProp: Divide the gradient by a running average of its recent magnitude. *COURSERA: Neural Networks for Machine Learning*, 2012.

Vikrant Varma, Rohin Shah, Zachary Kenton, János Kramár, and Ramana Kumar. Explaining grokking through circuit efficiency. *arXiv:2309.02390*, 2023.

Mitchell Wortsman, Peter J. Liu, Lechao Xiao, Katie Everett, Alexander Alemi, Ben Adlam, John D. Co-Reyes, Izzeddin Gur, Abhishek Kumar, Roman Novak, et al. Small-scale proxies for large-scale Transformer training instabilities. *ICLR*, 2024.

Liu Ziyin, Zhikang T. Wang, and Masahito Ueda. LaProp: Separating momentum and adaptivity in Adam. *arXiv:2002.04839*, 2020.